

DATABASE_SCHEMA_ANALYSIS.md

VSSC Infrastructure Management Portal

Database Schema Analysis & Data Model Reference

Version: 1.0

Last Updated: June 2026

PURPOSE OF THIS DOCUMENT

This document serves as the authoritative reference for all database entities used by the VSSC Infrastructure Management Portal.

Its purpose is to:

- Explain every table
- Explain every relationship
- Explain ownership mappings
- Explain infrastructure mappings
- Explain how analytics are derived
- Explain frontend implications

This document must be treated as the source of truth before implementing any frontend feature.

DATABASE OVERVIEW

The application operates on two separate MySQL databases:

```
proxmox_db  
ccds_db
```

DATABASE RESPONSIBILITIES

proxmox_db

Infrastructure Database

Stores:

- Clusters
- Nodes
- Storage
- Virtual Machines
- VM Ownership
- VM Placement
- Staff Directory

This database represents actual infrastructure state.

ccds_db

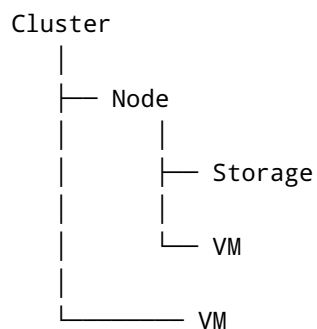
Application Database

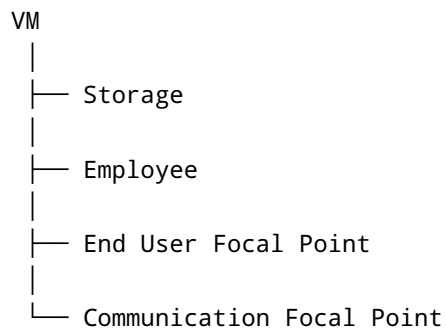
Stores:

- Authentication
- Roles
- VM Request Forms
- Requested VM Ownership

This database supports application workflows.

HIGH LEVEL ENTITY RELATIONSHIP





DATABASE 1: PROXMOX_DB

TABLE: cluster

Purpose

Represents a Proxmox cluster.

A cluster acts as the highest infrastructure grouping level.

All nodes belong to a cluster.

Primary Key

cluster_name

Columns

cluster_name

Type:

varchar(15)

Purpose:

Unique cluster identifier.

Example:

fsgpucluster-25

proxmox_token

Type:

varchar(50)

Purpose:

Authentication token used during synchronization.

Relationships

One Cluster



Many Nodes

One Cluster



Many Virtual Machines

(via vm_cluster_relation)

Frontend Usage

Used by:

- Dashboard
- Cluster Screen
- VM Inventory
- Analytics

TABLE: node

Purpose

Represents a physical hypervisor server.

A node hosts virtual machines.

Composite Primary Key

`cluster_name`
`node_name`

Columns

cluster_name

Foreign Key

References:

cluster.cluster_name

node_name

Node identifier.

Example:

`fsgpu1`

model

Hardware model.

total_mem

Memory available on node.

Units:

Bytes

total_cores

Available CPU cores.

hypervisor

Hypervisor version.

uptime

Node uptime.

ip

Management IP.

live_status

Boolean.

True:

Online

False:

Offline

Relationships

One Node

↓

Many Virtual Machines

One Node

↓

Many Storage Devices

Frontend Usage

Used by:

- Nodes Dashboard
 - Infrastructure Screen
 - Monitoring
 - Analytics
-

TABLE: storage

Purpose

Represents a storage resource attached to a node.

Examples:

- NFS
 - LVM
 - Local storage
-

Composite Primary Key

```
cluster_name
storage_name
node_name
```

Columns

storage_name

Storage identifier.

storage_type

Examples:

NFS
LVM
Directory

total_size

Capacity.

content

Storage content types.

storage_server_ip

Backend storage server.

storage_datastore

Datastore name.

live_status

Online/Offline.

Relationships

One Storage

↓

Many Virtual Machines

(via vm_storage_relation)

Frontend Usage

Used by:

- Storage Dashboard
 - VM Details
 - Capacity Analytics
-

TABLE: vm

Purpose

Central infrastructure asset table.

Represents every synchronized virtual machine.

Primary Key

uuid

Important Columns

uuid

Globally unique VM identifier.

vm_name

Human-readable name.

vm_host_name

Hostname.

vm_id

Proxmox VM identifier.

cpus

Allocated CPU count.

sockets

CPU socket count.

max_memory

Allocated RAM.

max_disk

Allocated storage.

os

Operating system.

Examples:

Windows
Ubuntu
RHEL
Rocky Linux

mac

Network MAC address.

ip

VM IP address.

status

Examples:

```
running
stopped
paused
```

uptime

Runtime duration.

gpu

GPU allocation count.

gpu_info

GPU metadata.

created_date

VM creation timestamp.

live_status

Boolean.

Current infrastructure presence.

request_source

Origin of VM request.

com_focal_point

Communication owner.

Foreign Key:

empdetails.staff_code

end_user_focal_point

End user owner.

Foreign Key:

empdetails.staff_code

dcv_hostname

DCV endpoint hostname.

display_type

Display configuration.

prometheus_status

Monitoring state.

software_installed

Installed application stack.

Relationships

One VM



One Cluster

One VM



One Node

One VM



Many Storage Devices

One VM



Many Users

Frontend Usage

Used almost everywhere.

TABLE: empdetails

Purpose

Staff directory.

Acts as the master employee dataset.

Primary Key

staff_code

Columns

staff_code

Unique employee identifier.

name

Employee name.

division

Organizational division.

groupname

Group identifier.

entity

Entity identifier.

Frontend Usage

Used by:

- Ownership assignment
 - Search
 - Analytics
 - Reports
-

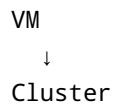
RELATION TABLES

TABLE: vm_cluster_relation

Purpose

Maps VMs to clusters.

Relationship



```
graph TD; VM --> Cluster;
```

VM
↓
Cluster

Usage

Supports:

- Cluster analytics
 - VM distribution charts
-

TABLE: vm_node_relation

Purpose

Maps VMs to nodes.

Tracks migrations.

Relationship

VM
↓
Node

Usage

Supports:

- Node dashboards
- VM placement analytics

TABLE: vm_storage_relation

Purpose

Maps VM disks to storage devices.

Relationship

VM
↓
Storage

Important Fields

vm_disk_image

size

initial_at

updated_at

Usage

Supports:

- Storage analytics
- VM details
- Capacity planning

TABLE: vm_user_relation

Purpose

Most important ownership table.

Maps VMs to employees.

Relationship

```
graph TD; VM --> Employee;
```

Many-to-Many

Usage

Supports:

- VM ownership
- Role filtering
- User dashboards
- Analytics

DATABASE 2: CCDS_DB

TABLE: user_role

Purpose

Application authentication.

Columns

staff_code1

role

password_hash

Current Roles

Original:

```
admin  
view_only
```

Frontend layer expands behavior into:

```
admin  
manager  
user
```

TABLE: vms

Purpose

Custom VM request submissions.

Important Columns

vm_name

cluster

ram

cores

os

disk_size

gpu

time_created

Usage

Supports VM request workflow.

TABLE: users

Purpose

Personnel associated with requested VMs.

Columns

staff_code

name

center

entity

groupname

division

section

TABLE: vm_users

Purpose

Many-to-many relationship.

Links:

Requested VM
↓
Personnel

COMPLETE OWNERSHIP MODEL

Infrastructure ownership is determined through:

vm_user_relation

This table drives:

- User access
- User dashboards
- Ownership analytics
- Resource accountability

ROLE FILTERING MODEL

Admin

No filtering.

Can view all VMs.

Manager

Read-only visibility.

Can view all VMs.

Cannot modify ownership.

User

Must only see:

```
SELECT *  
FROM vm  
WHERE uuid IN (  
    SELECT uuid  
    FROM vm_user_relation  
    WHERE staff_code = current_user  
)
```

This rule is critical.

ANALYTICS DERIVATION MODEL

The database supports numerous analytics without schema changes.

Infrastructure Analytics

Derivable:

- VM Count
 - Running VMs
 - Stopped VMs
 - Cluster Count
 - Node Count
 - Storage Count
-

Resource Analytics

Derivable:

- Total CPUs

- Total RAM
 - Total Storage
 - Total GPUs
-

Ownership Analytics

Derivable:

- VMs per Entity
 - VMs per Division
 - VMs per Group
 - CPU per Entity
 - RAM per Entity
 - Storage per Entity
-

Capacity Analytics

Derivable:

- VM Growth
- CPU Growth
- RAM Growth
- Storage Growth

using:

`created_date`

historical snapshots

FRONTEND IMPLICATIONS

Before implementing any feature:

Verify:

1. Required data exists.
2. Relationship exists.
3. API exposes relationship.
4. Feature does not require schema change.

If any feature requires a new table or endpoint, it must be rejected unless backend approval is obtained.

FINAL DATABASE ASSESSMENT

Strengths:

- Strong normalization
- Clear ownership model
- Excellent VM mapping structure
- Good analytics potential

Weaknesses:

- No dedicated manager mapping
- Limited audit logging
- Limited historical resource snapshots

Despite these limitations, the schema is sufficient to build a modern Infrastructure Operations Portal without database modifications.